CAMPUSVOTE

Implementation Roadmap
Ordered work remaining from DEVO.pdf

CURRENT STATUS

The starter Flutter app has a role launcher and a local-only ballot demonstration. The first Phase 1 task is implemented: election configuration parsing, validation, and a SHA-256 hash. This is a foundation only, not a production election system.

PHASE 1 — CORE (NO NFC)

1. Election configuration — DONE. Parse and validate positions, candidates, rules, voting window, trustee threshold, and voter-roll root. Hash the canonical JSON with SHA-256.
2. Voter roll and Merkle root — NEXT. Define roll input, validate it, and compute the Merkle root used by the locked election configuration.
3. Rust cryptographic core — Implement ElGamal ballot encryption, proofs, aggregation, threshold decryption, and tallying. Bind the Rust core to Flutter with flutter_rust_bridge; do not implement election cryptography in Dart.
4. Phase 1 verification — Add CLI/unit tests against the worked numbers in DEVO.pdf and property tests that compare encrypted tallies with plaintext counts.

PHASE 2 — BOOTH APPS AND BACKEND

Build the Rust/Axum API and PostgreSQL ledger. Implement election setup/lock, roll upload, card verification, blind-sign token issuance with the voted flag in the same transaction, ballot verification/submission, nullifiers, public ballot/receipt lookup, and trustee tally endpoints. Build distinct check-in desk and voting terminal apps with QR exchange, cast-or-spoil, receipt, and cleanup flows. Never store a link between voter identity and ballot.

PHASE 3 — NFC

Personalize NTAG 424 DNA cards. Implement Android APDU and Secure Dynamic Messaging validation, including CMAC and counter checks; reject UID-only cards and replayed/stale taps. Support card revocation and replacement.

PHASE 4 — SOLANA

Build the Anchor program and election, nullifier, batch-root, and booth-authority accounts. Lock the configuration before polls open; enforce nullifier uniqueness and anchor batch Merkle roots. Keep blind-signature verification in the backend.

PHASE 5 — HARDENING

Add offline booth operation with disjoint signed roll slices and an append-only hash-chained local log. Add kiosk/device integrity controls and revocable device keys, Benaloh cast-or-spoil challenges, optional thermal paper slips, fault-injection tests, an independent public-data verifier, and load tests.

PHASE 6 — PILOT

Run a small club-election dry run with volunteers and independent observers. Resolve pilot issues before a department-scale election. Obtain written election-committee approval before any real election.

CROSS-CUTTING CHECKS

Test duplicate/replayed tokens, cloned cards, duplicate ballot submissions, dropped connections, and tally proofs. Round/publish timestamps and ballots in shuffled batches to reduce timing correlation. Review the voter-roll and card-issuance trust assumptions. Design booth layout to reduce shoulder-surfing.

SCOPE AND SAFETY

This project is for supervised campus elections, not national or state elections. The demo UI is not secure or connected to real election services. Never use it to conduct a real election. The spec calls for Solana/Anchor; reconcile the current prototype's Ethereum/Sepolia placeholder before chain work begins.

IMPLEMENTATION UPDATE — 16 OCTOBER 2026

Phase 1, item 1: election configuration

Added lib/election_config.dart. The typed model reads the election JSON format in DEVO.pdf and serializes it back to JSON. Validation covers required fields and types, unknown fields, unique position and candidate codes, candidate counts versus seats, the plurality method, one vote per position, timezone-aware opening and closing times, trustee thresholds, and a 64-character hexadecimal voter-roll root.

The config hash is SHA-256 over the serialized JSON. Added the crypto package. This produces a local hash only; config locking on Solana is not implemented.

Added test/election_config_test.dart for config parsing and a fixed expected hash, JSON round-trip stability, malformed roots, duplicate positions, unknown fields, invalid or reversed voting windows, and invalid trustee thresholds. Updated README.md to describe the implementation and remaining work. Updated pubspec.yaml and pubspec.lock for the crypto dependency.

CHECKS RUN

Dart analysis completed with no issues. A runtime smoke check confirmed the expected hash, JSON round-trip, and rejection of a malformed root. The Flutter test runner stalled while starting its test device, before reporting test results. The added tests still need to be run successfully with flutter test.

STILL TO DO IN PHASE 1

Implement voter-roll validation and Merkle-root generation, then the Rust cryptographic core and tally. Add tests against the numerical examples in DEVO.pdf and property tests for randomized tallies.

Implementation Update

20 October 2026 · Voting-terminal UI

BALLOT PROGRESS GUIDANCE

Refined the local voting-terminal progress panel. Before all choices are made, it prompts the voter to choose one candidate for each position. After both choices are made, it shows a check icon and confirms the ballot is ready to review.

BEHAVIOR

The review action remains enabled only when both positions have a selection. Review is still a local demonstration: selections are not submitted or saved, and no election service or security feature is connected.

VALIDATION

Updated the voting-terminal widget test to check the initial guidance, the incomplete state after one selection, and the ready-to-review state after both selections. The existing review-dialog assertions continue to verify that selections are local only.

CAMPUSVOTE

Implementation Update

20 October 2026 · Ballot UI and project status

COMPLETED SINCE THE PREVIOUS UPDATE

Voting-terminal UI: added clearer selection progress and a “Clear selections” action. Clearing resets both positions; review remains unavailable until each required position has a choice. The review dialog continues to state that the selections are local and are not submitted or saved.

The earlier ballot UI work also added per-position candidate selection and a ready-to-review state. The role launcher and other workspaces remain demo placeholders.

Election configuration foundation: `lib/election_config.dart` parses and validates the typed configuration and computes its canonical JSON SHA-256 hash. This does not lock a configuration on-chain or implement the voter roll.

CHECKS AND CORRECTION

`flutter analyze` reports no issues and `flutter test` passes, including election-configuration and ballot widget tests. This supersedes the 16 October note on page 2 that the test runner had stalled; the tests have since completed successfully.

The app has been built and run in the Android emulator. The emulator run is a development smoke check, not a production-readiness assessment.

CURRENT COMPLETION BOUNDARY

Completed: starter UI, local-only ballot selection/review/reset, configuration parsing and validation, configuration hashing, and focused tests. Not completed: voter-roll processing, Merkle-root generation, Rust election cryptography, backend/API/database, NFC, Solana integration, or any production election workflow.

NEXT WORK — KEEP THIS ORDER

1. Phase 1: define and validate voter-roll input and generate the Merkle root exactly as specified in DEVO.pdf.
2. Phase 1: implement the security-critical cryptographic core in Rust, bind it to Flutter, and verify specification vectors and randomized tally properties.
3. Phase 2 onward: implement the backend, separate check-in/voting flows, NFC, Solana anchoring, hardening, and a supervised pilot, following the earlier roadmap pages.

SAFETY

This is still a prototype. Do not use it to conduct a real election. Keep election cryptography in the specified Rust core; do not imply that the current Flutter UI is connected to secure voting services.